

O&O Frontend 통합 개선 프롬프트

기능 버그 수정, 402×874 모바일 반응형, 데이터 사전 로딩, 체류시간 검증, 채팅 안정화, PhotoConfirmPage UI, 구조 개선, 접근성, 성능 및 레포 위생을 하나의 실행 명세로 통합한 문서

프로젝트	React + Vite + styled-components
기준 화면	402×874 모바일 UI
최우선	실제 기능 버그, 이벤트 중복, 라우팅, 채팅 메시지
검증	ESLint, format check, build, 핵심 사용자 시나리오

아래 내용을 프로젝트 작업 프롬프트로 사용한다. 단순히 lint 오류를 숨기는 것이 아니라 실제 사용자 흐름과 API 계약을 검증하며 운영 가능한 수준으로 개선한다.

1. 작업 원칙

- 기능이 실제로 깨지는 버그를 가장 먼저 수정한다.
- ESLint 규칙 비활성화나 예외 주석으로 문제를 숨기지 않는다.
- 기존 API endpoint와 payload를 추측으로 변경하지 않는다.
- 사용자 행동 이벤트의 누락과 중복을 모두 방지한다.
- 관련 없는 사용자 코드나 디자인을 되돌리지 않는다.
- UI 변경 중 기존 기능을 함께 삭제하지 않는다.
- styled-components는 별도 파일로 분리하고 페이지는 하위 컴포넌트 조합 중심으로 만든다.
- 로그에 방문 토큰, 사용자 사진, presigned URL 등 민감한 값을 노출하지 않는다.
- 각 단계의 변경 파일, 수정 이유, 위험도, 검증 결과를 기록한다.

2. 최우선 기능 버그

2.1 Analytics 라우트 오타

AnalyticsLoadingPage와 Router를 확인해 존재하지 않는 /analytics 이동을 제거한다.

- 정상 경로는 /analytics 또는 라우터에 정의된 slug 경로를 사용한다.
- slug 유무와 관계없이 빈 화면으로 이동하지 않게 한다.

완료 조건

- 코드 전체에서 /analytics 문자열이 사라짐
- 정상 Analytics 화면으로 이동

2.2 Analytics slug 전달 방식

Header는 /analytics-loading?slug=...를 전달하지만 AnalyticsLoadingPage가 useParams로 읽는 불일치를 제거한다.

- query string 유지 시 useSearchParams를 사용한다.
- URL의 slug를 우선하고 sessionStorage는 새로고침 복구용 fallback으로만 사용한다.
- URL만으로 분석 상태를 복구할 수 있어야 한다.

2.3 공유 화보 라우터 가드

실제 공유 주소 /lookbook/{shareSlug}는 방문 ID가 없는 외부 사용자도 접근할 수 있어야 한다.

- /lookbook/:shareSlug와 //:shareSlug를 공개 공유 경로로 처리한다.
- 그 외 매장 내부 페이지는 visit_id가 없으면 랜딩으로 이동시킨다.
- 공개 화보 조회 API의 인증 조건도 확인한다.

2.4 product_dwell 중복 전송

ProductPage의 useProductEvent, useDwellTimer, ProductInfo 프리셋 정산이 같은 이벤트를 중복 발송하는지 확인한다.

- 상품 페이지 전체 체류시간 전송 책임을 한 곳으로 통일한다.
- 프리셋 체류는 상품 전체 체류와 event type 또는 metadata subtype으로 구분한다.
- 뒤로가기, 라우트 이동, 관람 종료, 강제 flush에서 중복되지 않게 한다.
- Analytics 합계가 실제 체류보다 부족하지 않게 확인한다.

완료 조건

- product_view 1회
- 상품 전체 product_dwell 1회
- 마지막 체류시간 포함

2.5 setState updater 사이드 이펙트

ProductInfo의 handlePresetClick에서 updater 내부의 sendEvent, onQuestionClick, API, storage 변경을 제거한다.

- updater는 새 상태만 계산하는 순수 함수로 만든다.
- 이벤트와 콜백은 updater 밖에서 한 번만 실행한다.
- StrictMode에서도 이벤트가 중복되지 않게 한다.

2.6 Pretendard 폰트

Pretendard를 CSS에만 선언하고 실제로 로드하지 않는 문제를 해결한다.

- 로컬 폰트 또는 공식 CDN 중 한 방식을 사용한다.
- font-weight 매핑과 시스템 fallback을 설정한다.
- Figma와 실제 모바일 렌더의 글자 폭을 비교한다.

2.7 SSE 파싱 안정화

chat.js의 JSON.parse와 청크 결합 로직을 보강한다.

- JSON.parse를 try-catch로 보호한다.
- 불완전한 청크는 다음 청크와 합친다.
- 빈 줄, data prefix, 종료 이벤트, 비 JSON 이벤트를 구분한다.
- 한 줄의 오류가 전체 스트림을 종료시키지 않게 한다.
- AbortController와 reader cleanup을 확인한다.

2.8 메시지 ID 충돌

Date.now만으로 생성하는 로컬 메시지 ID를 안전하게 변경한다.

- 서버 메시지는 서버 고유 ID를 사용한다.
- 로컬 메시지는 crypto.randomUUID를 우선 사용한다.
- 텍스트나 타입만으로 dedupe하지 않는다.
- 빠른 연속 추가에서도 React key가 충돌하지 않게 한다.

3. 채팅과 상품 데이터

3.1 ABA 트리거 메시지

A → B → A에서 마지막 A가 사라지는 원인을 메시지 병합, key, pending action, polling 순서에서 찾는다.

- 동일 타입이라도 새로운 서버 메시지 ID면 표시한다.
- 동일 서버 메시지 ID를 재수신한 경우에만 중복 제거한다.
- pending_action은 해당 reply_to 메시지에만 연결한다.
- 늦은 polling 응답이 최신 메시지 배열을 덮지 않게 한다.

완료 조건

- A → B → A
- A → A
- B → A
- 동일 polling 응답 재수신

3.2 진열대 클릭 채팅 제거

진열대 클릭은 분석 이벤트로 남길 수 있지만 일반 채팅 메시지를 만들면 안 된다.

- scene_click과 hotspot_click 분석 이벤트는 요구사항에 맞게 유지한다.
- 진열대 클릭만으로 createChatMessage를 호출하지 않는다.
- 상품 클릭과 실제 챗봇 트리거는 유지한다.
- UI에서 텍스트를 숨기는 대신 생성 원인을 제거한다.

3.3 상품 상세 색상

상품 상세 색상은 product.attributes.color에서 가져온다.

- 실제 API 응답에서 문자열인지 배열인지 확인한다.

- 문자열은 trim하고 배열은 유효값을 join한다.
- 값이 없을 때만 색상 정보 없음을 표시한다.
- 다른 속성을 색상으로 대체하지 않는다.

```
const productColor = product?.attributes?.color;

const formatProductColor = (color) => {
  if (Array.isArray(color)) return color.filter(Boolean).join(", ") || "색상 정보 없음";
  if (typeof color === "string" && color.trim()) return color.trim();
  return "색상 정보 없음";
};
```

3.4 진열대 안내를 1~4 / 5~7 두 열로 배치

첨부된 기준 화면처럼 진열대 안내 목록은 왼쪽 열에 1~4, 오른쪽 열에 5~7이 세로로 표시되어야 한다. 현재 목록이 일렬로 나오거나 1:2 / 3:4 형태로 배치되는 원인을 FloorMap.jsx와 FloorMap.style.js에서 확인한다.

- ShelfInfoGrid의 styled-components 속성명이 실제 CSS로 출력되는지 확인한다. grid-templateColumns처럼 camelCase로 작성된 CSS 속성은 grid-template-columns로 수정해야 한다.
- 단순히 grid-template-columns: 1fr 1fr만 적용하면 DOM 순서상 1:2 / 3:4 / 5:6 / 7로 채워질 수 있으므로 기준 화면의 열 우선 배치를 명시한다.
- CSS Grid를 사용한다면 grid-auto-flow: column과 grid-template-rows: repeat(4, auto)를 조합하거나, 데이터를 [1,2,3,4]와 [5,6,7] 두 그룹으로 나눠 각각 렌더링한다.
- GUIDE_ITEMS의 실제 순서는 1 토트백, 2 백팩, 3 쇼퍼백, 4 악세서리, 5 여성 의류, 6 남성 의류, 7 F/W 신상을 유지한다.
- 제목은 목록 위 중앙에 유지하고, 두 열의 번호-텍스트 시작선과 행 간격을 첨부 화면에 맞춘다.
- 목록 텍스트가 한 줄로 길게 이어지거나 컨테이너 밖으로 넘치지 않게 한다.
- 320px부터 402px까지 지도 비율이 줄어들어도 오버레이와 목록이 함께 비례 축소되거나 내부 폭에 맞게 재배치되어야 한다.
- 안내 오버레이가 열려 있을 때 지도 zone 클릭이 실행되지 않고 닫은 뒤 정상 동작하는 기존 기능을 유지한다.

기준 정보 구조
진열대 안내

1. 토트백 5. 여성 의류
2. 백팩 6. 남성 의류
3. 쇼퍼백 7. F/W 신상
4. 악세서리

완료 조건

- 왼쪽 열은 1, 2, 3, 4 순서
- 오른쪽 열은 5, 6, 7 순서
- 각 항목은 한 줄로 표시되고 열 사이에 겹침이 없음
- 402×874와 320px 폭 모바일에서 첨부 기준 화면과 같은 정보 구조 유지

3.5 진열대 6→7 스와이프 이미지 플래시

진열대 6번에서 7번으로 스와이프할 때 7번 이미지가 전환 중 잠깐 먼저 튀어나오거나 잘못된 위치에 보이는 현상을 재현하고 원인을 수정한다. 지연 시간을 임의로 늘리거나 opacity로 순간적으로 숨기는 임시 처리보다 Swiper와

React 렌더링 타이밍의 실제 원인을 해결한다.

- Shelf.jsx의 Swiper에서 activeIndex, realIndex, urlZone, swiperZoneRef, selfWroteZoneRef가 6→7 전환 중 어떤 순서로 갱신되는지 기록한다.
- onSlideChangeTransitionEnd에서 selectShelf와 navigate를 실행할 때 route 변경으로 Swiper 또는 slide가 재초기화되는지 확인한다.
- urlZone effect의 slideTo(urlZone - 1, 0, false)가 transition 종료 직후 다시 실행되어 순간 점프를 만드는지 확인한다.
- SwiperSlide key는 zone처럼 안정적으로 유지하고, 내부 상품 이미지 key는 배열 index가 아니라 product_id 또는 고유 상품 ID를 사용한다.
- 6번 Shelf05와 7번 Shelf07이 서로 다른 레이아웃을 사용하므로 absolute positioning, overflow, transform, z-index가 인접 slide 밖으로 새는지 확인한다.
- Swiper viewport와 각 SwiperSlide에 overflow 경계를 명확히 하고, Shelf07의 이미지가 자신의 slide 범위 밖에서 그려지지 않게 한다.
- 7번 상품 이미지가 아직 decode되지 않은 상태에서 레이아웃이 바뀌는지 확인하고, 다음 zone의 실제 표시 이미지에 대해서만 preload와 decode를 적용한다.
- route 업데이트와 store selectShelf 업데이트가 동일 전환에서 중복 렌더를 만들지 않게 단일 책임으로 정리한다.
- Swiper 인스턴스를 key 변경으로 불필요하게 재생성하지 않으며, transition 중 중복 navigate를 막는다.
- 빠르게 5→6→7, 7→6, 6→7→6으로 왕복하는 경우와 느린 네트워크·캐시 비움 상태를 모두 검증한다.

완료 조건

- 6→7 전환 중 7번 이미지가 지정 영역 밖에 먼저 나타나지 않음
- 전환 종료 후 URL은 /shelf/7이고 Swiper index와 store zone이 모두 7로 일치
- 7→6 역방향과 빠른 연속 스와이프에서도 플래시·점프·빈 화면이 없음
- 이미지 캐시 유무와 네트워크 속도에 관계없이 레이아웃이 안정적임

4. 모바일 UI와 체류시간

4.1 402×874 모바일 반응형

402×874를 디자인 기준으로 사용하되 실제 모바일 뷰포트와 주소창 변화에 대응한다.

- 데스크톱에서는 최대 너비 402px 모바일 화면을 중앙 배치한다.
- 모바일에서는 실제 화면 너비와 100dvh, safe area를 사용한다.
- 기본 매장 화면은 앱 shell을 고정하고 채팅 메시지 영역만 스크롤한다.
- Analytics와 Lookbook의 긴 콘텐츠는 명시적인 내부 스크롤 영역을 사용한다.
- overflow hidden 중첩으로 콘텐츠가 잘리지 않게 한다.
- iOS에서 -webkit-overflow-scrolling: touch를 적용한다.
- 가상 키보드가 입력창과 최근 메시지를 가리지 않게 한다.

완료 조건

- 320×568

- 360×800
- 375×812
- 390×844
- 402×874
- 430×932

4.2 체류시간과 Analytics

상품, 장면, 진열대, 프리셋의 체류 시작과 종료를 전체 흐름에서 검증한다.

- 밀리초와 초 단위를 혼용하지 않는다.
- 하나의 체류를 여러 hook이 중복 전송하지 않는다.
- 관람 종료 직전의 마지막 체류시간을 포함한다.
- 서버 전송 완료 이벤트와 pending buffer를 중복 합산하지 않는다.
- 요청 payload와 Analytics 응답 매핑을 개발 환경에서 검증한다.

4.3 PhotoConfirmPage Figma UI

제공된 PhotoConfirmPage Figma node를 기준으로 UI를 구현한다.

- 간격, 색상, 폰트, 버튼, radius, 이미지 비율을 반영한다.
- 402×874와 작은 모바일 화면에서 잘리지 않게 한다.
- 다시 찍기, MediaPipe, timeout, mask fallback, presign, PUT, POST, 오류와 중복 제출 방지를 유지한다.
- API payload는 UI 작업 중 변경하지 않는다.
- styled-components는 PhotoConfirmPage.styled.js로 분리한다.

Figma: [PhotoConfirmPage Figma URL 및 node-id 입력]

5. 구조와 일관성

5.1 스토리지 키 통일

visitId, visit_id, visitToken, visit_token, anonymousUuid, anonymous_uuid 혼용을 정리한다.

- 키를 상수화하고 storage/auth 유틸을 만든다.
- 기존 세션은 migration fallback으로 읽는다.
- 새 저장은 표준 키 한 가지를 사용한다.
- API interceptor와 event API가 같은 값을 읽게 한다.

5.2 입장 경로 통일

LandingPage의 인자 없는 enterStore와 HomePage의 하드코딩된 성별/연령 호출을 하나로 정리한다.

- 사용자가 입력한 값은 그대로 전달한다.
- 값이 없으면 명세에 맞춰 생략한다.
- 테스트용 하드코딩 값을 제거한다.

- StrictMode에서도 /enter가 두 번 호출되지 않게 한다.

5.3 채팅 동기화 통합

Layout과 ChatPage에 중복된 `handleAction`, 3초 `polling`, 상태 복구 코드를 `useChatSync`로 추출한다.

- `polling interval`은 한 곳에서만 생성한다.
- 언마운트 시 `interval`과 요청을 정리한다.
- 동시에 여러 `polling` 요청이 겹치지 않게 한다.
- ABA 순서와 `pending action` 연결을 보존한다.

5.4 API와 Store 책임 분리

API 함수가 `storage`를 직접 읽거나 Store가 `styled-component`를 `export`하는 구조를 정리한다.

- API 함수는 필요한 값을 매개변수로 받는다.
- `storage` 접근은 공통 유틸로 분리한다.
- Zustand store는 상태와 action만 관리한다.
- 미사용 `MessageText` 등은 제거한다.

5.5 라우트와 데드 코드

중복 `Analytics route`, 호환 `Lookbook route`, `preview route`, `mocks`, 항상 `truthy`인 조건을 정리한다.

- `route path`를 상수화하고 `navigate`와 `Route`가 같은 값을 사용하게 한다.
- 개발 `preview route`는 개발 환경으로 제한한다.
- `drainEventBuffer ? ...` 같은 불필요한 조건을 제거한다.
- 삭제 전 실제 참조 여부를 검색한다.

5.6 데이터 사전 로딩과 캐시 전략

사용자가 다음 화면으로 이동하기 전에 필요한 데이터와 핵심 이미지를 미리 받아 전환 지연을 줄인다. 사전 로딩은 현재 화면의 첫 렌더와 사용자 입력을 막지 않아야 하며, 무조건 모든 데이터를 받는 방식이 아니라 실제 이동 가능성이 높은 다음 단계만 대상으로 한다.

- 입장 API 성공 후 `scenes`와 다음 지도·진열대 화면에 필요한 상품 요약 데이터를 준비한다.
- 지도에서 특정 `zone`을 가리키거나 선택하면 해당 진열대의 상품 데이터와 화면에 실제 표시될 이미지를 우선 `prefetch`한다.
- 상품 상세 진입 가능성이 높아진 시점에는 상세 데이터 요청을 미리 시작하되 동일 `product_id` 요청을 중복 호출하지 않는다.
- 관람 종료 응답으로 `report slug`를 받으면 `AnalyticsLoadingPage` 진입과 동시에 분석 데이터 조회를 시작한다.
- 화보 `job`이 `ready`가 되면 `LookbookPage`로 전환하기 전에 최종 `image_url`을 `preload`하고 이미지 `decode`가 끝난 뒤 결과 화면을 표시한다.
- `CameraPage` 진입 후 브라우저가 유휴 상태일 때 `MediaPipe` 모듈을 준비해 촬영 확인 화면의 대기시간을 줄인다.
- 요청 URL과 주요 파라미터를 `cache key`로 사용하고 진행 중인 `Promise`도 공유해 in-flight 요청을 합친다.
- 캐시에 TTL을 두고 방문 ID나 `report slug`가 바뀌면 관련 캐시를 무효화한다.

- sessionStorage는 새로고침 복구용으로 사용하고, 오래될 수 있는 서버 데이터의 유일한 source of truth로 사용하지 않는다.
- prefetch 요청 실패는 현재 화면의 정상 사용을 막지 않으며 실제 화면 진입 시 다시 요청할 수 있어야 한다.
- 화면을 벗어난 요청은 AbortController로 취소하거나 늦은 응답이 최신 상태를 덮지 않게 한다.
- 모바일의 navigator.connection.saveData, effectiveType, 느린 네트워크를 고려해 고용량 이미지와 MediaPipe 사전 로딩을 제한한다.
- 고해상도 이미지 전체를 무조건 preload하지 않고 실제 렌더 크기에 맞는 WebP 또는 AVIF 후보를 우선 사용한다.
- 방문 토큰, 사용자 사진 원본, presigned upload URL은 장기 캐시 또는 영구 저장소에 보관하지 않는다.
- 새 라이브러리를 추가하기 전에 현재 API 모듈에 작은 메모리 캐시를 적용할 수 있는지 검토하고, 상태 캐시 라이브러리는 중복 제거·무효화 이점이 명확할 때만 도입한다.

권장 흐름

입장 성공 → scenes 및 지도 핵심 데이터 준비
 zone 관심/선택 → 해당 진열대 상품 썸네일 prefetch
 상품 관심/클릭 → product_id 기반 상세 요청 dedupe
 finish 응답 → report slug 기반 Analytics 요청 시작
 job ready → 최종 이미지 preload-decode → Lookbook 표시

완료 조건

- 같은 endpoint와 key의 동시 요청이 한 번의 네트워크 호출로 합쳐짐
- prefetch가 실패해도 현재 화면과 실제 진입 요청은 정상 동작
- 방문 또는 slug 변경 후 이전 사용자의 캐시 데이터가 노출되지 않음
- 모바일 느린 네트워크에서 고용량 사전 로딩이 자동으로 제한됨
- 지도→진열대, 진열대→상품, 종료→Analytics, ready→Lookbook 전환 시간이 측정 가능하게 감소

6. UX, 접근성, 리소스

6.1 alert 통일

브라우저 alert를 O&O 세계관에 맞는 Toast 또는 Modal로 통일한다.

- 안내, 오류, 확인을 구분한다.
- aria-live와 포커스 이동을 처리한다.
- 비동기 요청 중 중복 표시를 방지한다.

6.2 로그와 개인정보

console 사용을 logger 유틸로 통합하고 프로덕션에서 불필요한 로그를 제거한다.

- 토큰, 사진 데이터, presigned URL, 전체 행동 payload를 출력하지 않는다.
- 오류 코드, endpoint 종류, status 같은 최소 정보만 남긴다.
- 중요한 오류 로그까지 무조건 제거하지 않는다.

6.3 OG 메타태그

공유 서비스에 맞게 title, description, og:title, og:description, og:image, twitter:card, favicon을 설정한다.

- SPA 정적 메타로 가능한 범위와 동적 공유 페이지가 필요한 범위를 구분한다.
- 카카오톡과 외부 공유 미리보기를 실제 확인한다.

6.4 접근성

GuideOverlay와 ExitModal을 포함해 키보드와 스크린리더 접근성을 개선한다.

- role=button 요소에 키보드 동작을 제공하거나 실제 button을 사용한다.
- 모달 포커스 트랩, Escape, 포커스 복귀를 구현한다.
- aria-modal, aria-labelledby, focus-visible, 버튼 이름을 확인한다.
- 빈 alt는 장식 이미지에만 사용한다.

6.5 MediaPipe 리소스

촬영마다 생성되는 SelfieSegmentation과 FaceDetection 인스턴스의 누수를 막는다.

- 성공, 실패, timeout 후 close를 호출한다.
- 늦게 완료된 작업의 결과를 무시하고 리소스를 정리한다.
- CDN 동적 로드와 package dependency 중 한 방식을 선택한다.

7. 컴포넌트, 성능, 레포 위생

7.1 페이지와 styled-components 분리

LookbookPage, LandingPage, AnalyticsPage, PhotoConfirmPage, ChatPage를 역할별 컴포넌트로 분리한다.

- 페이지는 데이터 조회, 상태 연결, 이벤트 연결, 하위 컴포넌트 배치를 담당한다.
- API는 src/api, 순수 변환은 src/utils, 복잡한 비동기는 src/hooks로 이동한다.
- 페이지 내부 styled-component와 정적 inline style을 별도 파일로 이동한다.
- 지나치게 작은 의미 없는 컴포넌트는 만들지 않는다.

```

Page
├── PageHeader
├── ContentSection
├── ResultSection
├── ActionSection
└── Modal
  
```

7.2 성능과 이미지

큰 PNG와 500KB 이상 JavaScript chunk를 분석하고 필요한 자산만 초기 로드한다.

- PNG를 WebP 또는 AVIF로 변환하고 필요하면 srcSet을 제공한다.
- 1MB 이상 이미지를 우선 최적화한다.
- route 단위 code splitting을 적용한다.
- MediaPipe는 필요한 페이지에서만 로드한다.
- 변환 전후 화질과 실제 렌더 크기를 비교한다.

7.3 레포 위생

src.zip, public/images.zip, output/imagegen, 추적 중인 .env와 임시 파일을 점검한다.

- Git 추적 상태를 먼저 확인한다.
- 민감한 .env가 커밋됐다면 secret 교체 필요성을 알린다.
- 사용자 원본 여부가 불명확한 파일은 임의로 삭제하지 않는다.
- .gitignore를 보완하고 삭제 항목과 복구 가능 여부를 보고한다.

7.4 포맷과 CI

Prettier와 EditorConfig를 도입하고 포맷 변경과 기능 변경을 가능한 한 분리한다.

- lint, format, format:check, build script를 제공한다.
- CI에서 npm ci, lint, format check, build를 실행한다.
- 테스트가 추가되면 CI에 포함한다.

```
"lint": "eslint ."  
"format": "prettier --write ."  
"format:check": "prettier --check ."  
"build": "vite build"
```

8. 필수 테스트 시나리오

모바일

- 320×568, 360×800, 375×812, 390×844, 402×874, 430×932
- 주소창 열림과 닫힘, safe area, 키보드, 긴 채팅, 팝업, 긴 결과 페이지

방문과 라우팅

- visit_id 없이 내부 페이지 접근
- visit_id 없이 공유 화보 접근
- Analytics slug query 복구와 존재하지 않는 slug

데이터 사전 로딩

- 동일 상품을 빠르게 반복 선택해도 네트워크 요청이 중복되지 않음
- prefetch 중 다른 zone으로 이동했을 때 오래된 응답이 현재 상태를 덮지 않음
- saveData 또는 느린 네트워크에서 고용량 이미지-MediaPipe 사전 로딩 제한
- 캐시 만료, 방문 변경, report slug 변경 후 최신 데이터 재조회
- prefetch 실패 후 실제 화면 진입 시 정상 재시도

채팅

- A → B → A, A → A, polling 재수신, pending action
- 진열대 클릭, 상품 클릭, 실제 질문, 긴 메시지 스크롤

지도 안내와 진열대 Swiper

- 진열대 안내가 왼쪽 1~4, 오른쪽 5~7 두 열로 표시
- 320px, 360px, 390px, 402px에서 목록 겹침과 한 줄 늘어짐이 없음
- 5→6→7, 6→7, 7→6, 6→7→6 전환
- 이미지 캐시 비움, 느린 네트워크, 빠른 연속 스와이프 상태
- 전환 종료 후 URL, Swiper index, store selected zone 일치

체류시간

- 상품 즉시 이탈, 5초 체류, 프리셋 열기와 닫기, 뒤로가기
- 관람 종료, 마지막 체류 flush, Analytics 합계 비교

촬영과 화보

- 카메라 허용과 거부, 다시 찍기, MediaPipe 성공과 실패와 timeout
- 사진 PUT, 마스크 PUT, 생성 POST 202, polling ready와 failed와 timeout
- share slug 새로고침 복구와 외부 공유 링크

접근성

- 키보드만 사용, 팝업 Escape, 포커스 복귀, 스크린리더 버튼 이름

9. 검증과 최종 보고

수정 후 반드시 다음 명령을 실행한다.

```
npm run lint
npm run format:check
npm run build
```

- 수정한 파일 목록과 문제별 원인
- 적용한 해결 방법과 API 계약 변경 여부
- 이벤트 형식 변경 여부와 제거된 중복 이벤트
- 수정 전후 ESLint 오류 수와 빌드 성공 여부
- 테스트한 시나리오와 남은 위험 요소
- 수동 확인이 필요한 백엔드 항목
- 삭제하거나 이동한 파일과 성능 개선 전후 수치

10. 실행 우선순위

1. 라우트, slug, 공유 링크처럼 실제 기능이 깨지는 버그
2. 체류시간과 이벤트 중복
3. setState updater 사이드 이펙트
4. ABA 메시지, SSE, 메시지 ID 충돌

5. 진열대 클릭 채팅 제거와 상품 색상 데이터
6. 진열대 안내 2열 배치와 6→7 Swiper 이미지 플래시
7. 모바일 레이아웃과 스크롤
8. 데이터 사전 로딩, 요청 중복 제거, 캐시 무효화
9. PhotoConfirmPage Figma UI
10. 스토리지, 입장, 채팅 동기화 구조
11. 접근성, 로그, 폰트, OG 메타
12. MediaPipe 리소스와 성능
13. 컴포넌트 분리, 포맷, CI, 레포 위생

11. 최종 실행 지시

위 우선순위에 따라 현재 코드를 먼저 분석하고, 기능 버그 수정, 구조 개선, UI 구현, 테스트 및 검증을 완료한다. 기존 API 계약과 사용자 흐름을 보존하며, 명백한 중복과 오류는 실제 원인을 해결한다. 작업 결과는 검증 명령과 사용자 시나리오의 증거를 포함해 보고한다.